SYSTEM CONFIGURATION (OPERATIONS & IMPLEMENTATION)

Smart Environment Monitor

This document describes how Smart Env Monitor is currently configured (Docker-based operation), the central settings, and how the components interact. The document is intended to be practical for handover to a municipality (IT/operations and business users).
For a pure user guide (how to use the system via the UI), see the file: doc/user_guide.md

1. COMPONENTS

The system consists of the following components:

- proxy (Caddy): Receives traffic on localhost:18080 and forwards it to the Django application.
- web (Django + Gunicorn): API + Dashboard + Admin.
- db (PostgreSQL): Stores sensor data, alert rules, alert events, and incidents.
- sensor/simulator.py: Simulates IoT flow by sending data from the dataset to the API.

2. DOCKER ENVIRONMENT (RECOMMENDED FOR OPERATIONS)

2.1 Start and stop

To start the system: docker compose up -d --build
To check status: docker compose ps
To shut down: docker compose down

2.2 URLs (local operation)

Dashboard: http://localhost:18080/dashboard/
Admin: http://localhost:18080/admin/
Health check: http://localhost:18080/health/
Sensor ingest (for simulator): http://localhost:18080/api/data/

3. ENVIRONMENT VARIABLES (.env)

The system is environment-driven. The .env file (not committed to version control) controls keys, database connection, rate limiting, and monitoring.
Important variables:

```
SECRET_KEY=...
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
DATABASE_URL=postgresql://smartenv:smartenv@db:5432/smart_env_monitor

PROXY_HTTP_PORT=18080
PROXY_HTTPS_PORT=18443
SITE_ADDRESS=:80
```

```
DEVICE_API_KEY=...
THROTTLE_SENSOR_INGEST=120/min
THROTTLE_SENSOR_READ=600/min

SENSOR_STALE_MINUTES=30
SENSOR_HEALTH_EMAIL_TO=

BACKEND_URL=http://localhost:18080/api/data/
SENSOR_INTERVAL=10
DATASET_PATH=sensor/dataset/environment_data.csv
```

Notes:

- DATABASE_URL must use the hostname db (not localhost) in the Docker environment.
- DEVICE_API_KEY is used both by the backend (to validate ingest) and by the simulator (to send the correct header).
- BACKEND_URL in Docker should normally be http://localhost:18080/api/data/.

4. DATA MODEL (CURRENT)

4.1 SensorReading

Sensor readings are stored in the SensorReading model with multiple measurements per row:

- sensor_id (e.g., beach name)
- timestamp
- water_temperature
- turbidity
- wave_height
- wave_period
- battery_life
- quality

Implementation is in api/models.py.

4.2 Alert rules and alert events

Alerts consist of:

- AlertRule: what to monitor (metric/operator/threshold) and where to send the alert.
- AlertEvent: log when a rule triggers (time, recipient, subject/body).

Alert rules are created and managed via Django Admin.

5. API (CURRENT)

5.1 Sensor ingest (IoT → backend)

- Endpoint: POST /api/data/
- Authentication: If DEVICE_API_KEY is set, it is required via either:
  - Header X-Device-Key: <DEVICE_API_KEY> or
  - Authorization: Bearer <DEVICE_API_KEY>
- Rate limiting: sensor_ingest (controlled via THROTTLE_SENSOR_INGEST)

5.2 Read API (dashboard / internal users)

Example calls:

- GET /api/data/latest/?sensor_id=<id>
- GET /api/data/history/?sensor_id=<id>&from=YYYY-MM-DD&to=YYYY-MM-DD

Rate limiting: sensor_read (controlled via THROTTLE_SENSOR_READ)

6. SENSOR SIMULATOR (sensor/simulator.py)

The simulator does the following:

- Reads the dataset from sensor/dataset/environment_data.csv.
- Sorts rows chronologically (based on Measurement Timestamp).
- Sends rows as JSON to BACKEND_URL with X-Device-Key if DEVICE_API_KEY exists in .env.
- Prints a clear link to the dashboard in the terminal on startup.

Starting the simulator:

```
python sensor/simulator.py
```

7. DASHBOARD AND ROLES (RBAC)

The dashboard is publicly readable at /dashboard/.

Login and roles are used to display and protect functions as follows:

- Admin: full access (including /admin/).
- Manager: report and incident functions.
- Analyst: analysis view.
- Viewer: read-only in the dashboard.

Roles are implemented via Django Groups and are checked in views and templates.

8. MONITORING (STEP 3)

8.1 Health endpoint

GET /health/ returns:

- 200 OK if web and database are working.
- 503 if the database is unreachable.

8.2 Sensor health (stale detection)

Two levels exist:

- UI: Dashboard shows OK/Stale per sensor based on SENSOR_STALE_MINUTES.
- CLI: The check_sensor_health command can be scheduled and can send email if sensors become stale.

Running the CLI command:

```
docker compose exec web python manage.py check_sensor_health --stale-minutes 30 --email-to drift@kommun.se
```

9. COMMON OPERATIONS COMMANDS

9.1 Create admin user

```
docker compose exec web python manage.py createsuperuser
```

9.2 View logs

```
docker compose logs web
docker compose logs proxy
docker compose logs db
```

9.3 Reset database (demo/test)

```
docker compose exec web python manage.py flush --no-input
```

10. DATASET

The dataset is located in the repository under sensor/dataset/environment_data.csv and is used by the simulator. The purpose is to simulate a realistic IoT flow from multiple sensors (beaches) to the dashboard.